# Computer Vision compact cheatsheet

## STEP 1 Seeing like a machine

### 1.1 What computer vision is and why it is hard

**The definition** Computer vision computes properties of the physical world from images and video. Its three jobs are measurement, perception and search.

**The task family** Classification, detection (what and where), categorisation, instance recognition, scene, attribute, activity and relationship recognition, and segmentation.

**Why it is hard** Illumination, viewpoint, scale, occlusion, clutter, intra class variation, inter class similarity, field of view, weather and messy environments.

**Robustness and generalisation** Robustness handles expected variation. Generalisation handles new or shifted data. Imbalance hides poor performance on rare cases.

**Size and cost** N = H × W × C, bytes = N × b ÷ 8, levels = 2 to the power b, raw video rate = frame bytes × fps. Systems trade accuracy against time, memory and energy.

**Classical and learned features** Classical features are designed by hand and are cheap and interpretable. Deep networks learn features in a hierarchy from edges to objects.

**Numbers in an image** $N = H \times W \times C$

**Raw video data rate**

$$\text{rate} = H \times W \times C \times \frac{b}{8} \times \text{fps}$$

EXAM Examiners like the pair detection versus recognition. Detection finds and localises objects. Recognition decides what they are and what they mean. Categorisation gives a class, instance recognition names one particular object.

TRAP A high score on the test set does not prove the model will work after deployment. If the deployment conditions are missing from the data, the test set cannot measure them.

TRAP Do not say that classical features such as HOG are needed "because CNNs overfit". The notes correct this. Overfitting depends on data size, model size, regularisation and augmentation. Classical features are useful because they are efficient, simple and easy to interpret.

TRAP OpenCV loads colour images in BGR order, not RGB. Most other libraries (Matplotlib, PIL, PyTorch models) expect RGB. Forgetting to convert is one of the most common bugs in vision code.

### 1.2 Marr's theory, edges and hand made features

**Marr's four stages** Input image, primal sketch (edges, blobs), 2½-D sketch (depth and orientation of visible surfaces), 3D model (full shape). 2½-D because it depends on the viewpoint.

**Good features** Distinctive, stable (repeatable) and invariant. Corners beat edges, edges beat flat regions. Detection finds points, description turns them into vectors.

**Gradients** Gx and Gy by differences or Sobel. Magnitude √(Gx² + Gy²), direction atan2(Gy, Gx), pointing across the edge from dark to bright.

**SIFT** DoG scale space for scale, dominant orientation for rotation, 4 × 4 × 8 = 128 descriptor, ratio test d1 ÷ d2 < 0.8 for matching.

**HOG** Cells of 8 × 8, 9 unsigned bins, votes weighted by magnitude, 2 × 2 block normalisation, 3,780 numbers for 64 × 128. Good for upright people, weak on pose changes and occlusion.

**Why classical features matter** Efficiency, interpretability, domain knowledge and support for deep pipelines, not because CNNs overfit.

**Gradient magnitude** $|\nabla I| = \sqrt{G_x^2 + G_y^2}$

**Gradient direction** $\theta = \text{atan2}(G_y, G_x)$

**Descriptor length** $4 \times 4 \text{ cells} \times 8 \text{ bins} = 128$

**Descriptor length** $(n_x - 1)(n_y - 1) \times 4 \times B$

EXAM Depth is not volume. A depth map gives the distance to each visible surface. A 3D model describes the whole object, including the hidden back. The exam asks this as "why is it called 2½-D".

EXAM Feature detection finds where the interesting points are. Feature description turns the neighbourhood of each point into a vector of numbers so points can be compared. SIFT does both. HOG is mainly a descriptor.

TRAP With image coordinates the y axis points down. An angle of 90° therefore means "brightness increases downwards". Many textbooks draw y upwards, so check which convention a question uses.

## STEP 2 Images as numbers

### 2.1 Pixels, colour models and compression

**Image as numbers** H × W × C values from 0 to 255. Check channel order, range, type, normalisation and layout before a model sees an image.

**Colour models** RGB adds light, CMYK adds ink, HSL separates hue from lightness, YCbCr separates brightness from colour. OpenCV uses BGR. Grey = 0.299R + 0.587G + 0.114B.

**Lossy and lossless** PNG keeps every pixel, JPEG throws away detail the eye misses. Use lossless for text, medical and legal images.

**JPEG in five steps** YCbCr, chroma downsampling, 8 × 8 DCT, quantisation (the lossy step), zig zag plus Huffman coding. DC = 8 × mean.

**I, P and B frames** I stands alone, P predicts from the past, B from past and future. B is smallest but adds delay, so live systems avoid it.

**Compression and evidence** Edge processing and content aware compression save bandwidth, but heavy compression can erase important detail.

**Stored values** $H \times W \times C$

**Grey from RGB (BT.601)**

$$Y = 0.299\,R + 0.587\,G + 0.114\,B$$

**Compression ratio** $\text{ratio} = \dfrac{\text{original size}}{\text{compressed size}}$

**2D DCT of an 8 × 8 block**

$$F(u,v) = \tfrac{1}{4}\,C(u)\,C(v)\,\Sigma_{x=0}^{7}\,\Sigma_{y=0}^{7}\,f(x,y)\cos\frac{(2x+1)u\pi}{16}\cos\frac{(2y+1)v\pi}{16}$$

**Quantisation** $F_q(u,v) = \text{round}\left(\dfrac{F(u,v)}{Q(u,v)}\right)$

TRAP Compression and normalisation are different things. Compression makes files smaller for storage and transfer. Normalisation changes the scale of numbers so a model can learn. The notes list this as a common confusion.

TRAP For medical, legal and industrial images, use lossless or very light compression. The fine edges that JPEG throws away can be the tumour boundary, the digit on a number plate or the crack in a weld.

### 2.2 Histograms, brightness and contrast

**Histogram** h(i) counts pixels with value i, p(i) = h(i) ÷ N, CDF is the running total. It tells how many, never where.

**Brightness, contrast, dynamic range** Brightness = mean (where the histogram sits), contrast = spread such as σ or Michelson, dynamic range = number of levels used.

**Point operations** Negative 255 − f, linear αf + β with clipping, log c·log(1 + r) to compress highlights, gamma 255(r ÷ 255)^γ, slicing for one band.

**Equalisation** s = round[(CDF − CDFmin) ÷ (1 − CDFmin) × 255]. Lecture example 80, 100, 120, 140 gives 0, 85, 170, 255. Not perfectly flat for discrete images.

**Why it matters** Dark, low contrast inputs weaken edges and bias models toward dominant regions. Use the same preprocessing in training and deployment.

**Cumulative distribution** $\text{CDF}(r) = \Sigma_{i \le r}\,p(i)$

**Brightness** $B(I) = \dfrac{1}{wh}\,\Sigma_{v=1}^{h}\,\Sigma_{u=1}^{w}\,I(u,v)$

**Negative** $g(x,y) = L - f(x,y)$

**Log transform** $s = c \cdot \log(1 + |r|)$

**Equalisation mapping**

$$s = \text{round}\left[\frac{\text{CDF}(r) - \text{CDF}_{\min}}{1 - \text{CDF}_{\min}} \times 255\right]$$

TRAP These methods change the input distribution. Apply exactly the same preprocessing at training time and at deployment, or the model sees two different worlds.

### 2.3 Noise, convolution and filtering

**Noise models** Additive g = f + n (Gaussian, salt and pepper), multiplicative g = f(1 + n) (speckle). Gaussian → averaging or Gaussian filter, impulse → median, speckle → adaptive or median type, shot → denoising.

**Convolution** Slide a kernel, multiply and add. Output size ⌊(W − k + 2P) ÷ S⌋ + 1. True convolution flips the kernel, correlation does not.

**Smoothing** Mean (1/9)·ones, Gaussian with σ (OpenCV σ = 1.7 for 9 × 9), median for outliers. Lecture patch: mean 23.33, median 10.

**Sharpening** Lecture high pass (1/9)[[−1, −1, −1], [−1, 8, −1], [−1, −1, −1]] sums to 0. Notebook kernel [[0, −1, 0], [−1, 5, −1], [0, −1, 0]] sums to 1 and equals identity minus Laplacian.

**Frequency view** g = h ∗ f ⟺ G = H · F. Low pass blurs, high pass keeps edges. Noise is added on purpose for robustness tests and is the core of denoising autoencoders and diffusion models.

**Additive noise** $g(x, y) = f(x, y) + n(x, y)$

**Multiplicative (speckle) noise**
$g(x, y) = f(x, y) \cdot (1 + n(x, y))$

**Filtering (correlation)**
$g(x, y) = \Sigma_{i=-a}{}^{a} \Sigma_{j=-b}{}^{b} h(i, j) \cdot f(x + i, y + j)$

**Output size** $O = \lfloor \frac{W - k + 2P}{S} \rfloor + 1$

**Mean (box) filter** $h = \frac{1}{9} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$

**Gaussian kernel**
$G(x, y) = \frac{1}{2\pi\sigma^2} \exp(-\frac{x^2 + y^2}{2\sigma^2})$

**Lecture high pass kernel**
$h = \frac{1}{9} \begin{pmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{pmatrix}$

**Notebook sharpening kernel**
$h = \begin{pmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix}$

**Convolution theorem**
$g = h * f \iff G(\omega_1, \omega_2) = H(\omega_1, \omega_2) F(\omega_1, \omega_2)$

EXAM Salt and pepper pushes a few pixels straight to an extreme by a sudden jump. Speckle changes values gradually by multiplying them. The mechanisms differ, which is why they need different filters.

TRAP Sharpening cannot bring back detail that blur destroyed. It boosts whatever high frequencies are left, and that includes noise. In the project, sharpening the blurred COCO128 images raised mAP50 from 0.6176 to 0.6277 but it stayed below the original 0.6706. You will meet those numbers again in the project lab.

## STEP 3 Motion and video

### 3.1 Optical flow, HOF, MBH and video tasks

**Optical flow** Flow (u, v) per pixel between frames T and T + 1. Magnitude √(u² + v²), direction atan2(v, u), speed = magnitude × fps. Lecture: (3, 4) gives 5 px per frame, 150 px per second at 30 fps.

**Sparse and dense** Sparse tracks a few corners and is cheap. Dense gives every pixel a vector and suits crowded scenes but costs much more.

**Brightness constancy** Ixu + Iyv + It = 0: one equation, two unknowns, so the aperture problem. Lucas and Kanade (window, least squares, corners) and Horn and Schunck (global smoothness).

**HOF and MBH** HOF bins flow directions (how things move). MBH bins gradients of the flow (where motion differs) and ignores uniform camera motion. HOG is appearance.

**Action pipeline** Interest points → patches → HOG, HOF, MBH → k-means visual words → histogram → SVM. Replaced by 3D CNNs, two stream networks and video transformers.

**Video tasks** Interpolation predicts a missing middle frame. Also stabilisation, denoising, re-identification, propagation over time, context graphs and video RAG.

**Flow magnitude** $|(u, v)| = \sqrt{u^2 + v^2}$

**Speed** speed = magnitude × fps

**Optical flow constraint** $I_x u + I_y v + I_t = 0$

**HOF bin weight**
$H[b] = \Sigma_{\text{pixels with direction in bin } b} \sqrt{u^2 + v^2}$

**MBH input**
$\nabla u = (\partial u/\partial x, \partial u/\partial y), \quad \nabla v = (\partial v/\partial x, \partial v/\partial y)$

**k-means objective** $J = \Sigma_i \|\mathbf{x}_i - \boldsymbol{\mu}_{c(i)}\|^2$

EXAM In flow pictures, colour shows direction and the strength of the colour shows speed. Learn to read the colour wheel, because exam figures often use it.

EXAM The rule from the lecture: whenever camera motion is present and could contaminate HOF, prefer MBH. When nothing competes with the motion you care about, HOF alone is sufficient.

TRAP Interpolation was defined in class only after several attempts. It is not generating new images and not editing pixels. It is predicting the missing frame between two frames.

## STEP 4

## Recognising what is in an image

### 4.1 Image classification and convolutional networks

**Classification** One label for the whole image, from a probability per class. ImageNet challenge: 1,000 classes, about 1.28 million training images. Hard because of inter-class similarity and intra-class variation.

**CNN layer** y = ReLU(W ∗ x + b), params k × k × $C_{in}$ × $C_{out}$ + $C_{out}$, output ⌊(W − k + 2P) ÷ S⌋ + 1. Pooling shrinks maps. Features go from edges to parts to objects.

**SoftMax and loss** $p_i = e^{z_i} \div \Sigma e^{z_j}$, loss $-\ln p_{true}$, gradient p − y. [2, 1, 0.1] gives [0.659, 0.242, 0.099]. Sigmoids for multi-label.

**Confidence** Confidence is usually the SoftMax output. Inductive bias (2, 4, 6) shapes it. A calibrated model is right c of the time when it says c.

**Architectures** AlexNet (GPU, ReLU, augmentation), VGG (stacked 3 × 3), Inception (several sizes, 1 × 1), ResNet H(x) = F(x) + x, ViT (patches as tokens).

**Design axes** Architecture, loss, optimisation. 1 × 1 conv on 80 × 80 × 128 with 256 filters gives 80 × 80 × 256 and 33,024 parameters.

**One convolution layer**
$y = \text{ReLU}(W * x + b), \quad \text{ReLU}(t) = \max(0, t)$

**Parameters of a layer**
$\text{params} = k \times k \times C_{\text{in}} \times C_{\text{out}} + C_{\text{out}}$

**SoftMax** $p_i = \frac{e^{z_i}}{\Sigma_j e^{z_j}}$

**Sigmoid** $\sigma(z) = \frac{1}{1 + e^{-z}}$

**Cross entropy loss** $L = -\Sigma_j y_j \ln p_j = -\ln p_{\text{true}}$

**Calibration**
Among all predictions made with confidence $c$, the fraction that is correct should be about $c$

**Residual learning** $H(x) = F(x) + x$

**Gradient descent** $w \leftarrow w - \eta \frac{\partial L}{\partial w}$

EXAM Classification gives one label for the whole image. Detection adds a box for each of a variable number of objects. Segmentation labels every pixel. Know all three outputs.

TRAP Some models now report lower error than the estimated human error on ImageNet. That does not mean human level vision. Models still fail under unusual conditions, distribution shift, adversarial changes, occlusion, new categories and ambiguous scenes. Benchmark wins are specific to the benchmark.

## STEP 5 Finding objects

### 5.1 From sliding windows to anchor boxes

**Detector output** For each object: class, box, confidence. Corners (x1, y1, x2, y2) or centre size (xc, yc, w, h). The number of objects varies.

**Sliding windows** (⌊(W − w) ÷ s⌋ + 1)(⌊(H − h) ÷ s⌋ + 1) windows per shape: 3,285 for 64 × 128 at stride 8 in 640 × 480. Four problems: location, scale, shape, computation.

**Anchors and regression** Rough box plus four learned corrections. t = ((x − xa) ÷ wa, (y − ya) ÷ ha, ln(w ÷ wa), ln(h ÷ ha)), decode with x = xa + tx wa and w = wa e^tw.

**Heads and anchor design** One backbone pass, conv heads score all anchors. k(C + 1) class and 4k box channels. 9 anchors = 3 scales × 3 ratios. SSD 8,732 boxes. Stride s ≤ 0.367w.

**Trusting scores** Aleatoric (data noise) versus epistemic (model ignorance). SoftMax is valid but often overconfident. ECE measures it. Conformal prediction gives a coverage guarantee.

**Two box formats**
$(x_1, y_1, x_2, y_2) \iff (x_c, y_c, w, h)$

**Number of window positions**
$N = (\lfloor \frac{W - w}{s} \rfloor + 1) \times (\lfloor \frac{H - h}{s} \rfloor + 1)$

**Encode (make the training target)**
$t_x = \frac{x - x_a}{w_a}, \quad t_y = \frac{y - y_a}{h_a}, \quad t_w = \ln \frac{w}{w_a}, \quad t_h = \ln \frac{h}{h_a}$

**Decode (turn a prediction into a box)**
$x = x_a + t_x w_a, \quad y = y_a + t_y h_a, \quad w = w_a e^{t_w}, \quad h = h_a e^{t_h}$

**Head sizes**
class channels = $k(C + 1)$, box channels = $4k$

EXAM Overlap between windows is a consequence, not a separate problem. A fine stride makes neighbouring windows overlap heavily, and that is why the same pixels are processed again and again.

### 5.2 Training a detector: targets and losses

**Target assignment** Match anchors to true boxes by IoU. RetinaNet: ≥ 0.5 positive, < 0.4 background, between ignored. RPN: 0.7 and 0.3. Background gets only a class loss.

**Box loss** MSE squares errors (outliers dominate), MAE is robust but kinked, Huber = ½e² for |e| ≤ 1 else |e| − ½, gradient clip(e, −1, 1). Bad box example: 345× under e², 1.1× in gradient with Huber.

**Imbalance** Easy background swamps the loss: 97.8% in the worked example. Remedies: subsample, down-weight, hard example mining, focal loss.

**Cross entropy and focal loss** BCE = −ln $p_t$. FL = $-\alpha(1-p_t)^\gamma \ln p_t$, γ = 2, α = 0.25. Background share falls to 0.54%. Bias $b = \ln(\pi \div (1-\pi)) = -4.595$ for π = 0.01.

**Design axes** Architecture, loss, optimisation. GAN → DCGAN (architecture) → WGAN (loss, earth mover distance).

**Per-anchor loss**

$$L = \frac{1}{N} \Sigma_{\text{anchors}} [\, \alpha\, L_{\text{cls}} + \beta\, \delta(\text{positive})\, L_{\text{loc}} \,]$$

**Huber (Smooth L1)**

$$\text{Huber}(e) = \tfrac{1}{2}e^2 \text{ if } |e| \le 1, \quad |e| - \tfrac{1}{2} \text{ otherwise}$$

**Binary cross entropy**

$$\text{BCE} = -[\, y \ln p + (1-y)\ln(1-p) \,]$$

**Focal loss**

$$\text{FL}(p_t) = -\alpha_t\,(1-p_t)^\gamma \ln p_t$$

**Prior bias**

$$\sigma(b) = \pi \implies b = \ln \frac{\pi}{1-\pi}$$

### 5.3 IoU, NMS and detector architectures

**IoU** |A ∩ B| ÷ (|A| + |B| − |A ∩ B|). Lecture: 0.389 (miss) and 0.747 (hit) at 0.5. Equal squares shifted by s: (1 − s) ÷ (1 + s).

**NMS** Sort by score, keep the best, remove same class boxes with IoU above the threshold, repeat. Slide example gives {A, D}. Crowds and fixed thresholds are weak spots. Soft-NMS decays by $e^{-\text{IoU}^2 \div \sigma}$.

**One and two stage** One stage: SSD, YOLO, RetinaNet. Two stage: Faster R-CNN with a class agnostic RPN. The speed and accuracy divide is fuzzy today. Stage wise, joint and hybrid training.

**RetinaNet and FPN** ResNet + FPN + focal loss + prior bias. Early layers: high resolution, low meaning. FPN: 1 × 1 lateral + ×2 upsample + add, shared heads. 76,725 anchors at 640 × 640.

**RoIAlign** Bilinear f = Σ weights × corners, weights are opposite areas and add to 1. (2.3, 4.6) gives 26.8. RoIAlign samples bins without rounding, so the detector trains end to end.

**IoU**

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}$$

**One FPN merge**

$$P_l = \text{Conv}_{1\times1}(C_l) + \text{Upsample}_{\times2}(P_{l+1})$$

**Bilinear interpolation**

$$f(x, y) = (1-u)(1-v)\, f_{00} + u(1-v)\, f_{10} + (1-u)v\, f_{01} + uv\, f_{11}$$

TRAP The lecturer mentioned comparing with the ground truth. At test time there is no ground truth, so NMS uses only scores and box to box IoU. Ground truth is used in training (target assignment) and in evaluation.

### 5.4 Evaluating and choosing detectors

**Matching** Sort by score. TP if IoU above α with an unmatched true box, otherwise FP (poor box, background or duplicate). Unmatched true boxes are FN. No true negatives. Three dogs example: TP 2, FP 2, FN 1.

**Precision and recall** P = TP ÷ (TP + FP), R = TP ÷ (TP + FN), F1 = 2PR ÷ (P + R). Example 0.5, 0.667, 0.571. A higher score threshold usually raises P and lowers R.

**AP** Envelope $p_{\text{interp}}(r)$ = max precision at recall ≥ r. All point AP adds rectangles where recall grows. Lecture list gives 0.674, and the 11 point version also 0.674. COCO uses 101 points.

**mAP and COCO** mAP = mean AP over classes. COCO AP also averages IoU 0.50 to 0.95, so 0.80 at IoU 0.5 became 0.559. $AP_S$ below 32², $AP_L$ above 96².

**Practice** Dominated models are never the right pick. Lower resolution and fewer proposals speed things up. Batch 32 to 2 makes BN 4 times noisier, so use frozen BN, GroupNorm or Sync-BN. Pre-training helps: 34.3 to 36.7 to 37.4. Long tail: r = max(1, √(t ÷ f)).

**Repeat factor sampling (LVIS)**

$$r(c) = \max(1, \sqrt{t \div f(c)}), \quad r(\text{image}) = \max \text{ over its classes } r(c)$$

**Precision**

$$\text{P} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

**Recall**

$$\text{R} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

**Interpolated precision**

$$p_{\text{interp}}(r) = \max_{r' \ge r} p(r')$$

**AP, all points**

$$\text{AP} = \Sigma_k\, (r_k - r_{k-1}) \cdot p_{\text{interp}}(r_k)$$

**mAP**

$$\text{mAP} = \frac{1}{C} \Sigma_c\, \text{AP}_c$$

**COCO AP**

$$\text{AP}_{\text{COCO}} = \frac{1}{10} \Sigma_{t \in \{0.50,\, 0.55,\, \ldots,\, 0.95\}} \text{mAP}_t$$

EXAM Remember the names this way. The first word says whether the model was right, the second says what it predicted. A false positive is a box the model drew that was wrong. The lecturer added that books draw the confusion matrix in different layouts, so learn the idea and not one picture.

TRAP Check which metric a paper reports. COCO AP numbers of 30 to 50 look small next to PASCAL VOC AP@0.5 numbers of 70 to 90, but they measure a much stricter thing.

## STEP 6 Understanding every pixel

### 6.1 What segmentation predicts

**The task** $I \in \mathbb{R}^{H\times W\times 3} \to Y \in \{1..C\}^{H\times W}$. Before argmax H × W × C, after argmax H × W. 512 × 512 needs 262,144 predictions. Use the cheapest task that answers the question.

**Semantic, instance, panoptic** Semantic: class per pixel, cannot count touching objects. Instance: object identity. Panoptic: both, scored by PQ = SQ × RQ.

**SoftMax per pixel** $p_c = e^{z_c} \div \Sigma\, e^{z_k}$. argmax of SoftMax = argmax of logits. SoftMax for one label per pixel, sigmoids for multi-label. Ignore label 255 gets no loss.

**Why classifiers fail** Convolution is equivariant, global pooling is invariant. Downsampling grows the receptive field (r += (k − 1)dj) but erases thin and small structures. What the encoder drops, the decoder cannot recover.

**WHAT and WHERE** Early layers: high resolution, WHERE. Deep layers: semantics, WHAT. Every architecture in the next chapter combines the two. Same body, different head for depth, flow and keypoints.

**Semantic segmentation**

$$I \in \mathbb{R}^{H\times W\times 3} \to Y \in \{1, \ldots, C\}^{H\times W}$$

**SoftMax at each pixel**

$$P(c \mid x, y) = \frac{e^{z_c(x,y)}}{\Sigma_k\, e^{z_k(x,y)}}$$

**Receptive field recurrence**

$$r_l = r_{l-1} + (k-1)\cdot d_l \cdot j_{l-1}, \quad j_l = j_{l-1} \cdot s_l$$

EXAM The slide warned: do not memorise architecture names. FCN, U-Net, DeepLab and transformers are different answers to one question: how do we combine WHAT is present with exactly WHERE it is?

### 6.2 Segmentation architectures

**FCN** An FC layer equals a conv with the map size as kernel, so classifiers become dense predictors. FCN-32s, 16s and 8s predict at 1/32, 1/16 and 1/8 and fuse earlier maps.

**Upsampling** Nearest copies, bilinear blends (0, 25, 75, 100 for the lecture map), transposed conv learns. $H_{\text{out}} = (H_{\text{in}} - 1)s - 2p + k$. Use k = 2s to avoid checkerboards. Masks: nearest only.

**Encoder-decoder and skips** Encoder: what. Decoder: where. Bottleneck: strongest context, coarsest detail. Skips carry WHERE. Concat costs about twice addition (73,792 against 36,928).

**U-Net** Contracting path, bottleneck, expanding path, skip at every level. Original 572 → 388 with valid convs, 31.0 M parameters, about half in the bottleneck.

**DeepLab** Dilated 3 × 3: $k_{\text{eff}} = 2r + 1$, 9 weights, padding r. Rates 1, 2, 4, 8 give a field of 31. ASPP: rates 6, 12, 18 + 1 × 1 + pooling. v3+ adds a decoder at stride 4.

**Output size**

$$H_{\text{out}} = (H_{\text{in}} - 1)\cdot s - 2p + k + p_{\text{out}}$$

**Dilated 3 × 3 kernel**

$$k_{\text{eff}} = k + (k-1)(r-1) = 2r + 1$$

TRAP When the kernel size is not divisible by the stride, some output cells receive more overlapping contributions than others, which prints a checkerboard pattern into the mask. Fixes: kernel = 2 × stride (k = 4, s = 2), or resize with nearest or bilinear and then apply a normal convolution, or use sub-pixel convolution (PixelShuffle).

EXAM Teaching sentence from the slide: deep features say this region is probably a car, skip features decide the car boundary is exactly here.

EXAM The class answered context for road against sidewalk confusion, and both slide decks agree. The recording has the lecturer saying boundary, which is most likely a transcription slip. In exams: road and sidewalk confusion is a context problem.

### 6.3 Training and evaluating segmentation

**Pixel loss** L = −(1 ÷ |Ω|) Σ ln p over valid pixels. Ignore label 255 leaves both the sum and the count. P(car) 0.9, 0.5, 0.05 gives 0.105, 0.693, 2.996.

**Imbalance** Weights $w_c = N \div (C\, n_c)$: pedestrian 20 in the street image. Focal $(1 - p_t)^\gamma$ scales by difficulty. Soft Dice 1 − 2Σpg ÷ (Σp + Σg), usually with CE.

**Metrics** Accuracy hides rare classes (90% with tumour IoU 0). Dice = 2TP ÷ (2TP + FP + FN), IoU = TP ÷ (TP + FP + FN), Dice = 2IoU ÷ (1 + IoU). mIoU from a dataset confusion matrix.

**Pipeline** Same spatial transform for image and mask, colour only for the image. Masks resize with nearest neighbour. Fit statistics on the training split. Split by patient, scene or video.

**Debugging** Six failures: boundary, small or thin, class confusion, domain shift, occlusion, over-smoothing. Check alignment, IDs, ignore handling, imbalance and per class IoU before blaming the architecture.

**Pixel-wise cross-entropy**

$$L = -\frac{1}{|\Omega|} \Sigma_{i \in \Omega} \ln p_{i, y_i}$$

**Weighted cross-entropy**

$$L = \frac{\Sigma_i\, w_{y_i}\, (-\ln p_{i, y_i})}{\Sigma_i\, w_{y_i}}$$

**Soft Dice loss** $L_{\text{Dice}} = 1 - \dfrac{2\,\Sigma\, p_i\, g_i + \varepsilon}{\Sigma\, p_i + \Sigma\, g_i + \varepsilon}$

**Dice** $\text{Dice} = \dfrac{2|P \cap G|}{|P| + |G|} = \dfrac{2\text{TP}}{2\text{TP} + \text{FP} + \text{FN}}$

**IoU (Jaccard)**

$$\text{IoU} = \frac{|P \cap G|}{|P \cup G|} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}}$$

TRAP Very large weights can make training unstable or raise false positives. Weights can follow importance, not only frequency.

TRAP Benchmarks add one confusion matrix over the whole dataset, then compute IoU per class. Averaging per image IoUs is different and noisier, since an image without a class gives 0 ÷ 0. Say which one you report.

EXAM Leakage in segmentation also comes from slices of one patient or frames of one video landing in both train and test. Split by patient, scene or video, never by random image. A model with high training IoU and low validation IoU is probably overfitting.

## STEP 7 Vision meets language and 3D

### 7.1 Connecting vision and language

**CLIP** Image and text encoders into one space. Cosine similarity. Symmetric contrastive loss over an N × N matrix with temperature τ. 400 million pairs, batch 32,768.

**Zero-shot** One prompt per class, pick the highest similarity, SoftMax of 100 × cosine for probabilities. Prompt wording matters and ensembling helps. CLIP gives scores, SAM gives masks.

**Transformers** Patch = token, (n ÷ p)² tokens, N² attention pairs: 196 → 38,416, 1,024 → 1,048,576. Windows cut the cost (64× in the Swin example). SegFormer, Mask2Former queries with Hungarian matching.

**SAM and language** SAM: ViT encoder, prompt encoder, light decoder, SA-1B (11 M images, 1.1 B masks), class agnostic. Language-guided segmentation returns a mask for a phrase. Referring segmentation answers with a mask, VQA with text.

**Generation, restoration, 3D** DALL-E makes images from text, CLIP matches them. Restoration and super-resolution infer plausible detail, so validate clinically. Stereo Z = fB ÷ d, error ≈ Z² Δd ÷ (fB).

**The processing ladder**

pixels → edges and textures → shapes → objects → scenes and activities

**Cosine similarity** $\cos(x, y) = \dfrac{x \cdot y}{\|x\|\, \|y\|}$

**Symmetric contrastive loss**

$$L = \tfrac{1}{2}\,[\, \text{CE}_{\text{rows}}(S \div \tau) + \text{CE}_{\text{columns}}(S \div \tau)\,]$$

**Tokens** $N = (n \div p)^2$

**Stereo depth** $Z = \dfrac{f\,B}{d}$

TRAP Some models now beat the estimated human error on the ImageNet benchmark. The lecturer's caution: that is task specific. The same models can still fail under unusual conditions, distribution shift, adversarial changes, occlusion and new categories. Benchmark superiority is not general intelligence.

EXAM CLIP and SAM both take prompts but answer different questions. CLIP says what an image corresponds to and outputs scores. SAM says which pixels belong to the object and outputs masks.

TRAP SAM is class agnostic. It tells you where an object is, not what it is. Pair it with a detector or CLIP when you need names.

TRAP A visually attractive restored image can still contain wrong information. In medicine, legal evidence and inspection, invented detail is dangerous, so validate restoration on the task it serves, not on how it looks.

## STEP 8 Project lab

### 8.1 Project lab: blur, sharpening and detection

**The experiment** Frozen YOLO11n on COCO128 (128 images, 929 objects). Only the pixels change: baseline, 9 × 9 Gaussian blur, then the sharpening kernel. Labels are copied because nothing moves.

**Blur** GaussianBlur with σ = 0 uses σ = 0.3 ((k − 1) ÷ 2 − 1) + 0.8 = 1.7 for k = 9. Separable, 18 multiplications instead of 81. A low pass filter that removes edges and texture, hurting small and thin objects most.

**Sharpen** [[0, −1, 0], [−1, 5, −1], [0, −1, 0]] = identity minus Laplacian = unsharp masking with a = 4. Weights add to 1. Gain 3 − 2 cos ω along an axis. Clipped to 0 to 255, which makes halos.

**Results** Blur: P 0.6633 → 0.6802, R 0.5894 → 0.5358, mAP50 0.6706 → 0.6176, mAP50-95 0.5026 → 0.4434. Sharpening won back about 20 percent of the recall and mAP losses and dropped P to 0.6583.

**Limits** COCO128 overlaps the training data, 128 images is a small sample, P and R come from the max F1 confidence, and one blur level was tested. Sharpen only images known to be blurred, and consider training with blur augmentation.

**Gaussian weight** $G(x, y) \propto e^{-(x^2 + y^2) \div (2\sigma^2)}$

**σ chosen by OpenCV**

$$\sigma = 0.3\,((k - 1) \div 2 - 1) + 0.8$$

**The project kernel**

$$K = \begin{pmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix} - \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

**Share of the loss won back**

$$\frac{\text{restored} - \text{degraded}}{\text{baseline} - \text{degraded}}$$

EXAM Your analysis found exactly this. In 000000000531.jpg detections fell from 10 to 4 and the tennis racket and distant players disappeared. In 000000000260.jpg detections fell from 8 to 3. Blur also caused class confusion: in 000000000049.jpg a horse was labelled as a dog with score 0.30.

TRAP filter2D computes correlation, not convolution, so the kernel is not flipped. For symmetric kernels such as this one the two are identical. The border default reflects the image without repeating the edge pixel (BORDER_REFLECT_101).

EXAM A careful conclusion uses several metrics and says which one decides. mAP summarises every threshold, so it is the main score for detectors. P and R describe one operating point.

## STEP 9 Exam revision

### 9.1 Exam revision: from school to research universities

**Image size in bytes**

width × height × channels × bytes per value

**Colour to grey**

$$Y = 0.299\,R + 0.587\,G + 0.114\,B$$

EXAM In a numerical, write the formula, substitute, then compute. Most lost marks come from skipping the first line, so the examiner cannot give method marks when the arithmetic slips.

TRAP A common slip is mixing bits and bytes. 24 bits per pixel is 3 bytes per pixel.

TRAP Write the box format before computing IoU. (x1, y1, x2, y2) and (x, y, w, h) give very different answers from the same four numbers.

EXAM Trap options to expect: forgetting the bias in a parameter count, using k instead of k – 1 in the receptive field, counting the background class in mIoU when the question excludes it, and rounding in the middle of a calculation.

EXAM In derivations, name each assumption when you use it: brightness constancy, small motion, rectified cameras, known σ. Examiners often ask what breaks when an assumption fails.